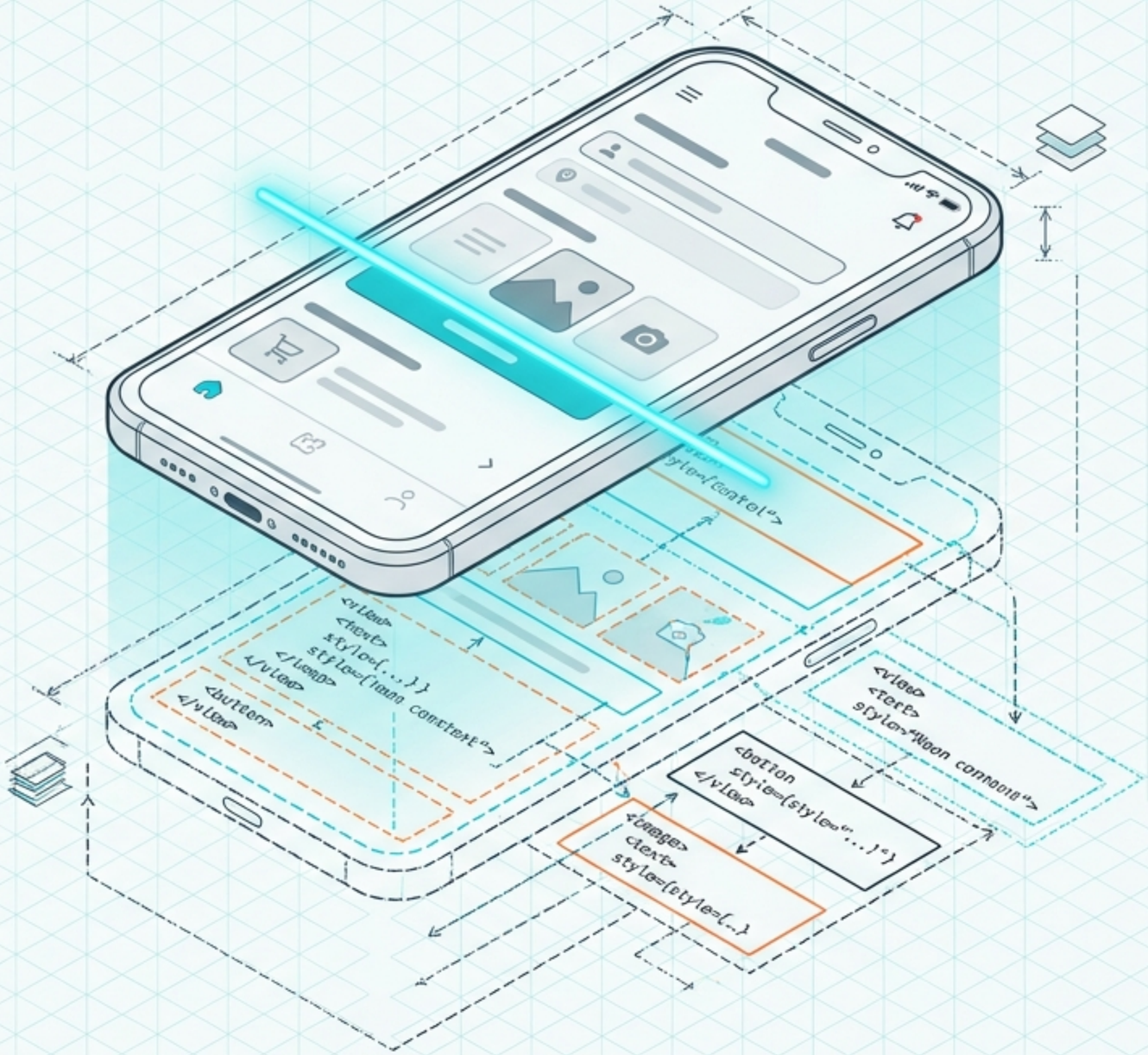
DeclarUI : L'Anatomie de l'Interface

Dépasser les limites des MLLMs par la vision par ordinateur, la logique de graphe et le raffinement agentique.

Une architecture systémique pour la génération de code UI production-ready.



NotebookLM

L’illusion de la magie MLLM

Les **modèles de langage multimodaux** (Claude, GPT-4V) excellent en théorie, mais **échouent en pratique face aux interfaces réelles.** Ils traitent les interfaces comme des compositions visuelles statiques plutôt que comme des systèmes interactifs complexes.



Le résultat ?
Un code qui semble correct mais s’effondre à l’usage.



NotebookLM

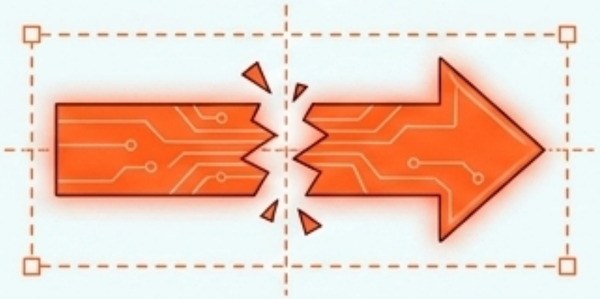
Matrice Diagnostique : Les 4 murs de l'IA générative brute

Reconnaissance Imprécise



Omission d'éléments critiques et hallucinations visuelles dus à la complexité des designs bruts.

Cécité Logique



Incapacité à déduire les flux de navigation à partir d'images statiques.

Incohérence Multi-Pages



Des comportements de navigation divergents pour un même bouton sur différents écrans.

Instabilité du Code



Génération de syntaxe non compilable et violation des conventions des frameworks.

NotebookLM

Le diagnostic en pratique : L'échec sur Lazada

Intention du designer

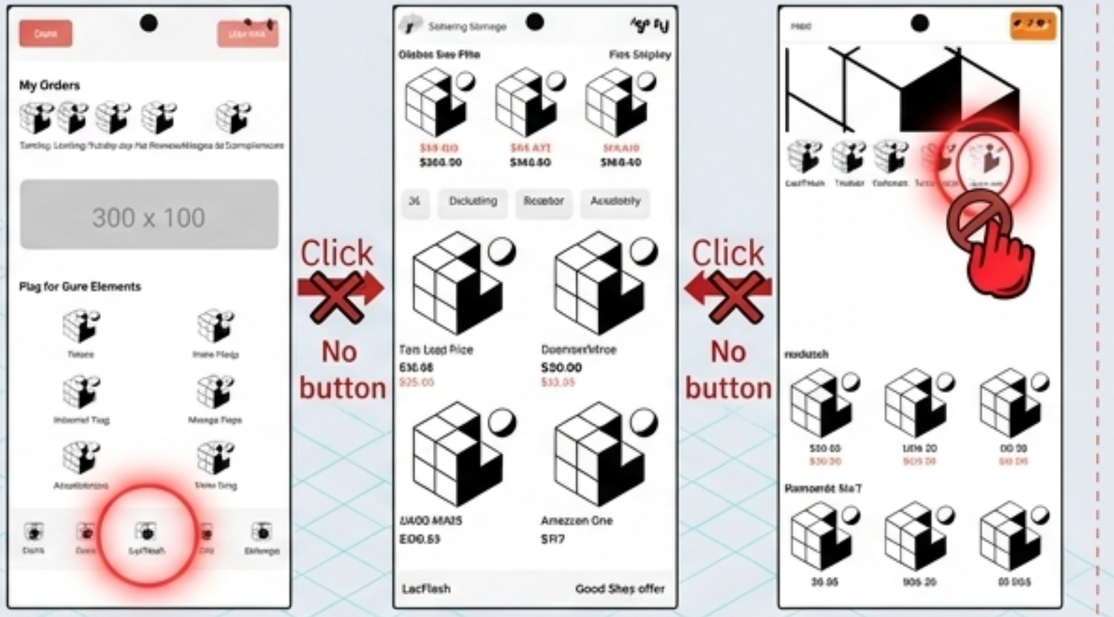


(a1) Account page (a2) LazFlash page (a3) Home page

Intention du designer :
Le bouton 'LazFlash' connecte la page Compte à la page Promo.

VS

Rendu Claude-3.5



(b1) Account page (b2) LazFlash page (b3) Home page

Rendu Claude-3.5 :
Bouton purement et simplement oublié. La logique de transition globale de l'application est détruite.

NotebookLM

L’Architecture DeclarUI : De l’image au système



Extraction Sémantique
(Vision Layer)

Grounding DINO + SAM.
Cartographie géométrique
des pixels.

Graphe de Navigation
(Logic Layer)

Modélisation PTG
(Page Transition Graph).
Le passage au comportement 4D.

Boucle d'Auto-Réparation
(Agentic Layer)

Validation des transitions et
correction par le compilateur.

NotebookLM

Phase 1 : L'Écorché de la Vision (CV Layering)



Segmentation (SAM)

Masques d'instance au pixel près.
L'IA délimite avec une précision chirurgicale.

Détection (Grounding DINO)

Génération de bounding boxes sans entraînement spécifique à l'UI (Zero-shot).
L'IA localise.

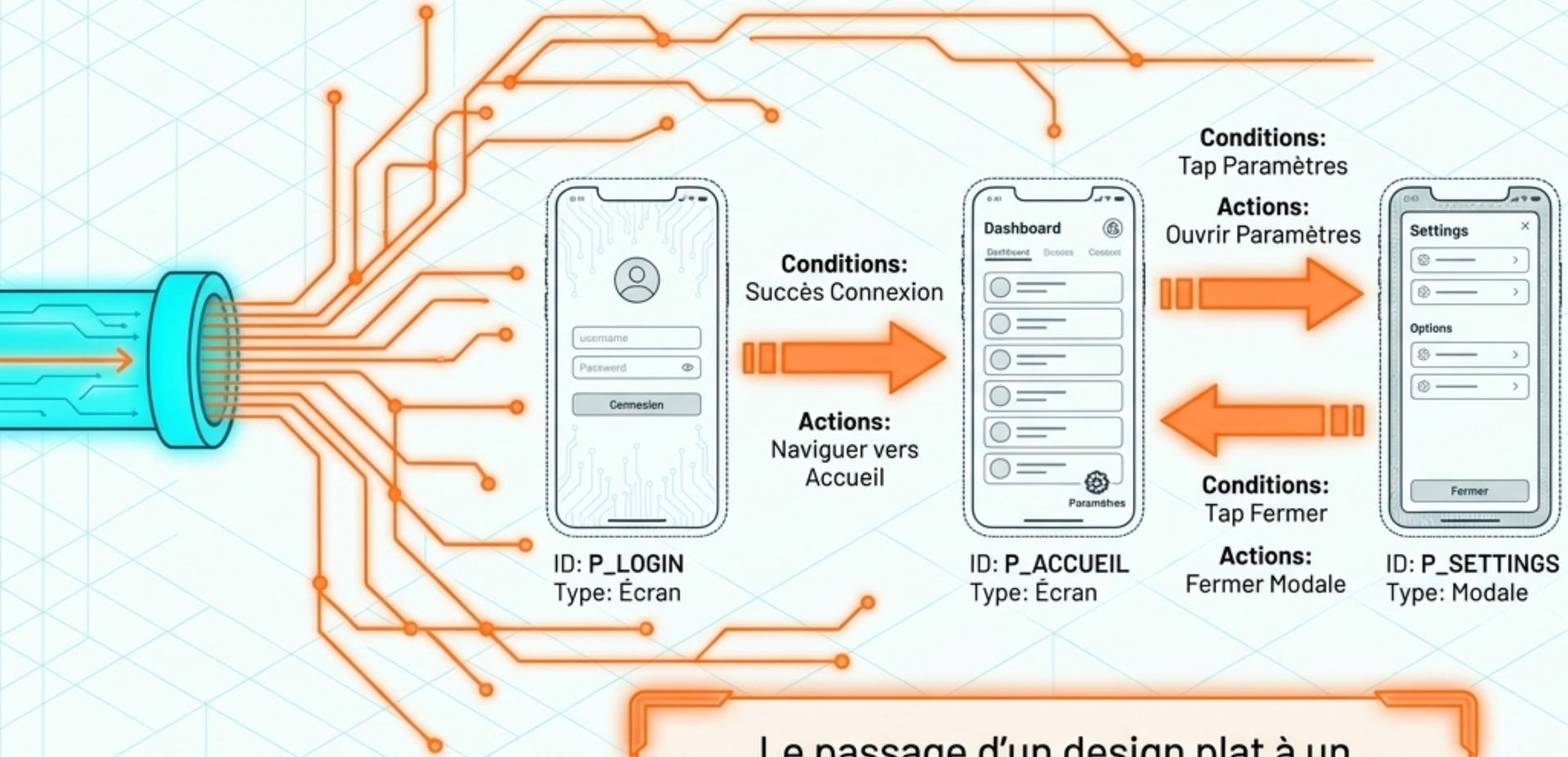
Pixels Bruts

L'image statique fournie par le designer.

L'IA ne devine pas l'interface, elle la cartographie géométriquement.

NotebookLM

Phase 2 : Le Graphe de Transition (PTG)



Une interface n’est pas qu’une collection d’écrans isolés.

DeclarUI extrait un graphe structuré (PTG) qui dicte les règles de navigation globales.

Le passage d’un design plat à un comportement dynamique inter-pages.

NotebookLM

Phase 3 : Le Raffinement Itératif (L'Agent Autocorrecteur)

Génération initiale du code basée sur la Vision et le PTG.





Test de compilation réel (Android Studio / DevEco).





Extraction chirurgicale des erreurs (fichier, ligne, syntaxe) transformées en prompts de correction.

DeclarUI ne se contente pas d'écrire du code. Il le compile, le teste et le répare comme le ferait un développeur humain (jusqu'à 3 itérations).

NotebookLM

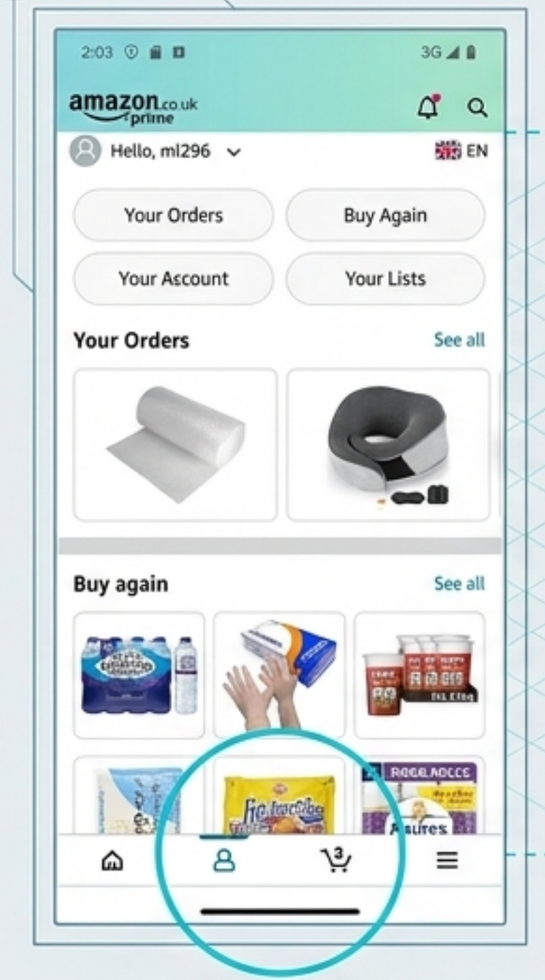
L'Anatomie de l'Échec vs La Rigueur du Système

Métrique	Approche Traditionnelle (Claude-3.5)	Système DeclarUI
Fidélité Visuelle (SSIM)	0.41	**0.68** (+65%) ↑
Couverture Logique (PCR)	43.3%	**96.8%** (Le gouffre de la compréhension)
Succès de Compilation (CSR)	74.0%	**98.0%** (Production-ready) ✅
Itérations Moyennes	Échec définitif ❌	**0.22** (Efficacité redoutable)

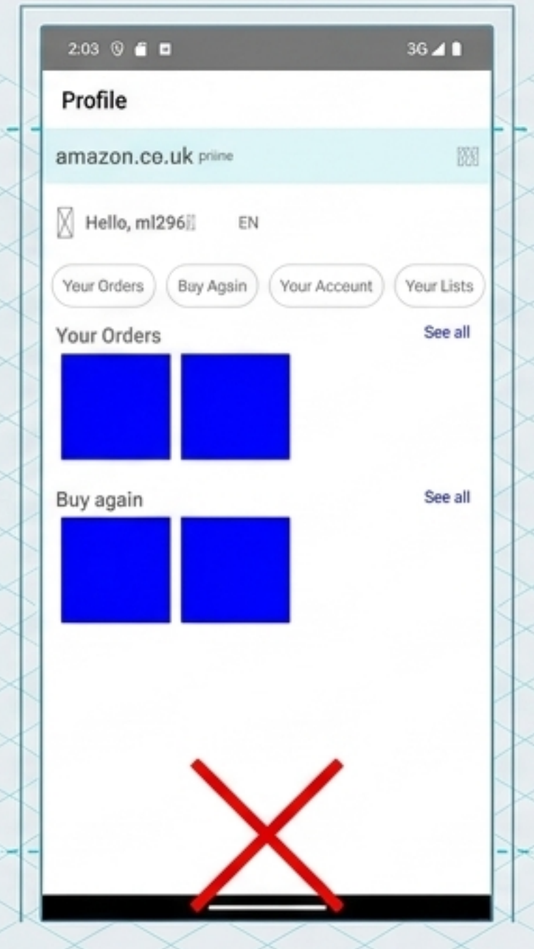
Évaluation sur React Native. Les données prouvent que l'architecture en couches résout les limites inhérentes des MLLMs.

SSIM
PCR
SSIM
CSR
NotebookLM

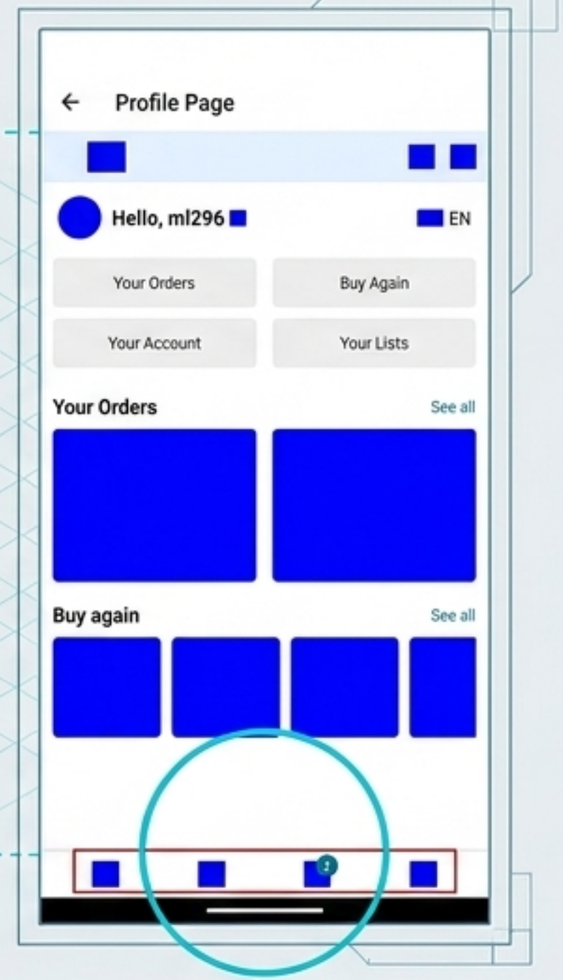
La Preuve Visuelle : Le Gap Amazon



Design Original.
Interface complexe avec barre de navigation inférieure.



Baseline.
L'IA classique hallucine la disparition du menu inférieur. Le composant est perdu.



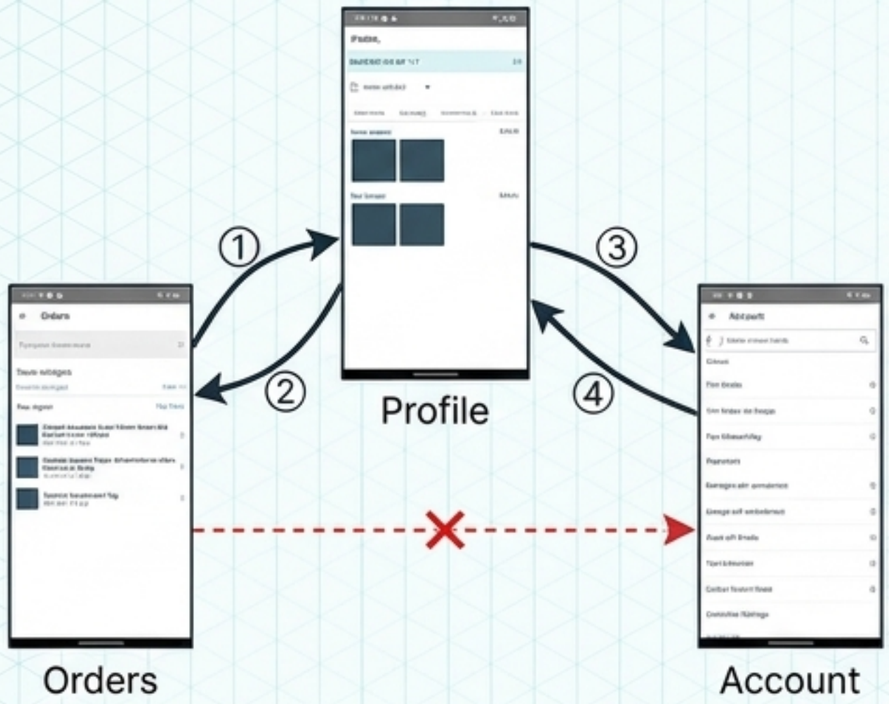
DeclarUI.
Copie structurelle parfaite. La barre de navigation est détectée et codée avec précision grâce au pré-traitement Vision (DINO+SAM).

NotebookLM

La Preuve Logique : Cohérence Multi-Pages

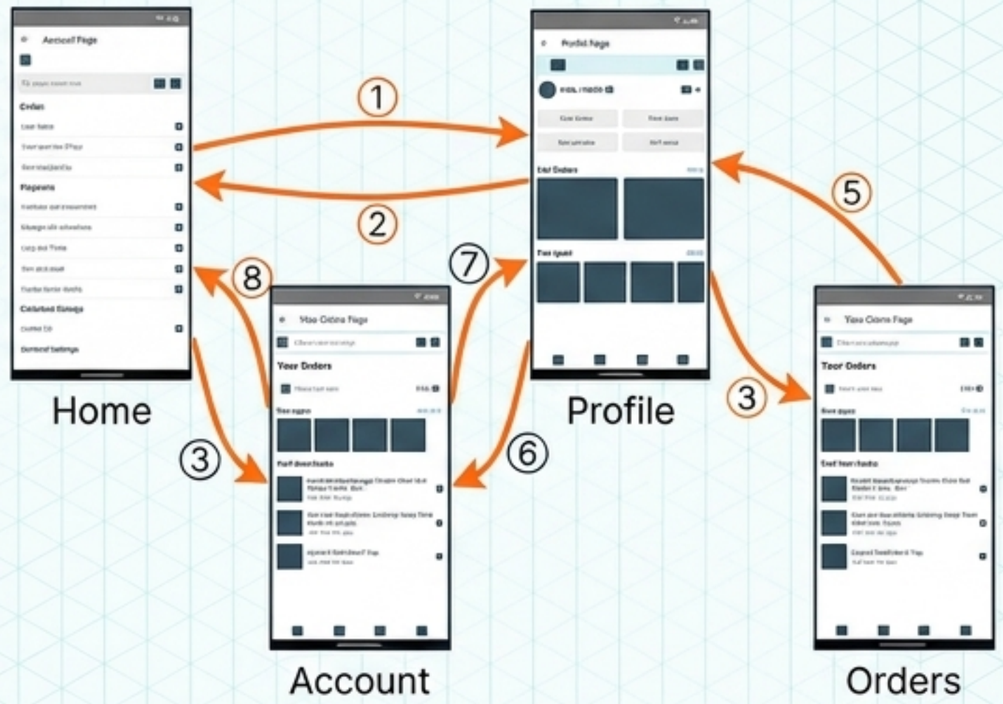
L'impact du Graphe de Transition (PTG) sur le flux utilisateur (User Transition Graph - UTG).

Flux Baseline (PTG Original)



Flux Baseline : Les transitions vers 'Orders' depuis le compte ou le profil sont oubliées. Expérience utilisateur brisée.

Flux DeclarUI (PTG Complet)



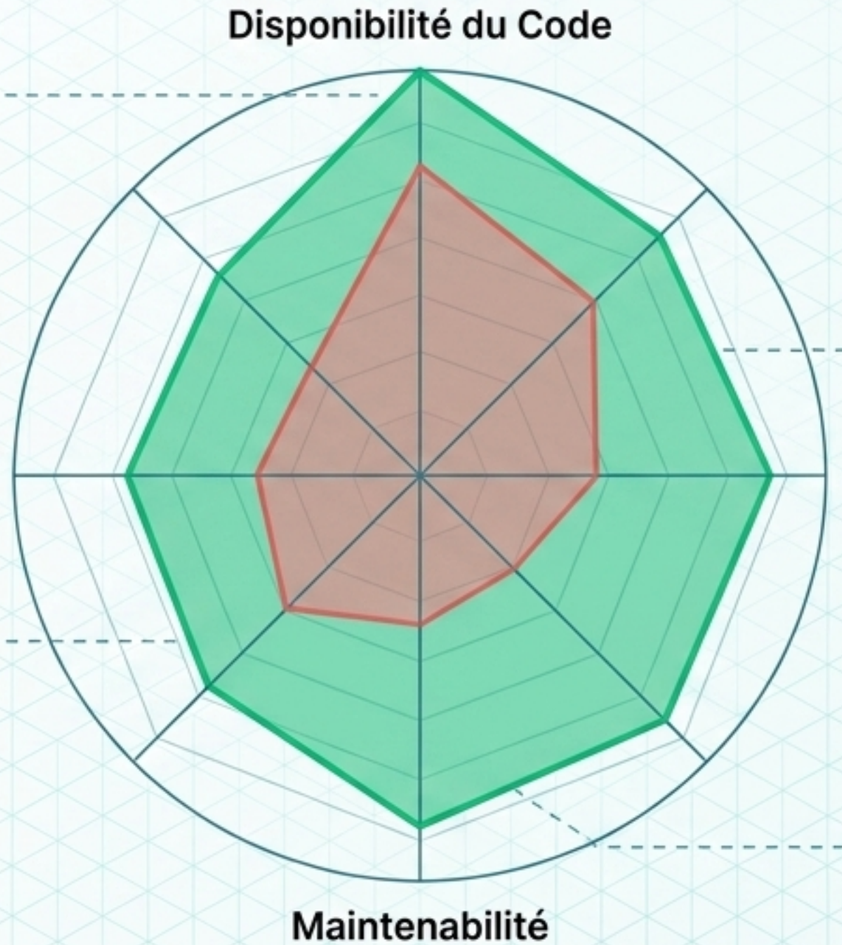
Flux DeclarUI : Tous les chemins de navigation croisés sont préservés. Le graphe garantit une cohérence à l'échelle de l'application.

NotebookLM

La Validation Humaine : Prêt pour l'Industrie

Étude menée auprès de développeurs professionnels React Native (Note sur 5).

Disponibilité du Code (4.97 vs 4.15).
Proportion de code directement utilisable.



Lisibilité (4.62 vs 3.75).
Compréhension de l'architecture générée.

Temps de Modification (4.03 vs 2.45).
Temps drastiquement réduit pour atteindre le standard de production (48s en moyenne).

Maintenabilité (4.55 vs 3.37).
Facilité d'adaptation aux nouveaux besoins métiers.

NotebookLM

Agnosticisme Technologique : Diagnostic Multi-Frameworks

React Native

Couverture PTG : **96.8%** |
Succès : **98.0%**

L'excellence opérationnelle.

Flutter

Couverture PTG : **99.4%** |
Succès : **92.0%**

Compréhension quasi-parfaite de la navigation.

ArkUI

Couverture PTG : **95.9%** |
Succès : **86.0%**

Opportunité Stratégique : Pour les frameworks émergents (ArkUI), l'injection d'un graphe de connaissances (RAG) réduit déjà le temps de correction manuelle de 75%.

NotebookLM

Le futur du flux de travail UI

DeclarUI transforme l'intention visuelle en une réalité fonctionnelle, sans compromis sur la rigueur d'ingénierie.



Conception & Maquette Statique

Traitement DeclarUI (IA & Logique)

Compilation & Génération de Code

Déploiement & UI Fonctionnelle

- **Intégration Transparente :** De la maquette au pipeline CI/CD.

- **Réconciliation :** Les designers conçoivent, l'agent traduit, les développeurs orchestrent.

"Le code généré n'est plus un brouillon instable, c'est un blueprint fonctionnel et validé."

NotebookLM